



Università della Calabria

**Dipartimento di Ingegneria Informatica, Modellistica,
Elettronica e Sistemistica**

Master Degree in ‘Robotics And Automation Engineering’

Course: Programmazione dei Sistemi Tempo-Reali e Distribuiti

***Reti di Petri, Timed Automata e Scheduling: Analisi e
Casi di Studio***

Student

Alessandro Furina
252328

Professor

Francesco Pupo

Academic Year 2024-2025

Indice

Introduzione

<i>1. Reti di Petri</i>	<i>1-8</i>
<i>1.1 Caso di Studio: CNC Machine</i>	<i>4-8</i>
<i>2. Timed Automata</i>	<i>9-16</i>
<i>2.1 Caso di Studio: Gestione Diga</i>	<i>9-16</i>
<i>3. Real-Time Scheduling</i>	<i>17-24</i>
<i>3.1 Rate Monotonic (RM)</i>	<i>18-21</i>
<i>3.2 Earliest Deadline First (EDF)</i>	<i>21-22</i>
<i>3.3 Least Slack Time (LST)</i>	<i>23-24</i>

INTRODUZIONE

Il seguente elaborato si occupa di analizzare dei semplici casi di studio inerenti alla progettazione di sistemi. In prima istanza, viene posta attenzione sulla fase di modellazione. Di fatti, prima ancora di testare un sistema è possibile dedurre dal suo modello importanti proprietà sia riguardo il corretto funzionamento che le tempistiche di esecuzione. Quest'ultime ricoprono un ruolo essenziale nei cosiddetti sistemi Real-Time, ovvero processi informatici in cui i tempi di risposta hanno una forte influenza sulla correttezza. In particolare, è possibile classificare i sistemi in:

- **Hard Real-Time:** processi in cui i tempi di risposta sono cruciali, il mancato rispetto può gravemente compromettere il sistema. Esempi di ciò sono dati da: sistemi di controllo di centrali nucleari o veicoli aerei, settore militare.
- **Soft Real-Time:** processi in cui i tempi di risposta pongono dei vincoli sul funzionamento pur, tuttavia, non precludendone l'utilizzo. Possibili esempi sono: processi all'interno dei videogiochi, sistemi di controllo base come la climatizzazione di ambienti o veicoli.

Con il seguente lavoro si vogliono analizzare due possibili approcci per ottenere un modello in grado di descrivere coerentemente il comportamento di un fenomeno o sistema, ovvero le Reti di Petri e gli Automi a Stati Finiti. Successivamente, verrà descritto il problema dell'organizzazione dei processi pronti per l'esecuzione analizzando differenti tecniche di scheduling.

1. Reti di Petri

Le Reti di Petri sono un potente strumento descrittivo in grado di rappresentare l'evoluzione di un sistema o processo sulla base delle operazioni che possono modificarne lo stato interno. Formalmente, una Rete di Petri è caratterizzata dalla seguente quintupla:

$$P/T = (P, T, F, W, M_0)$$

In cui:

- ***P*** è un insieme finito di posti.
- ***T*** un insieme finito di transizioni.
- ***F*** la relazione di flusso.
- ***W*** la funzione che assegna pesi non negativi agli archi.
- ***M₀*** la marcatura iniziale.

Con:

$$P \cap T = \emptyset, P \cup T \neq \emptyset$$

$$F \subseteq (P \times T) \cup (T \times P)$$

$$W: F \rightarrow N^+$$

$$M_0: P \rightarrow N$$

Un semplice esempio di rete è di seguito riportato:

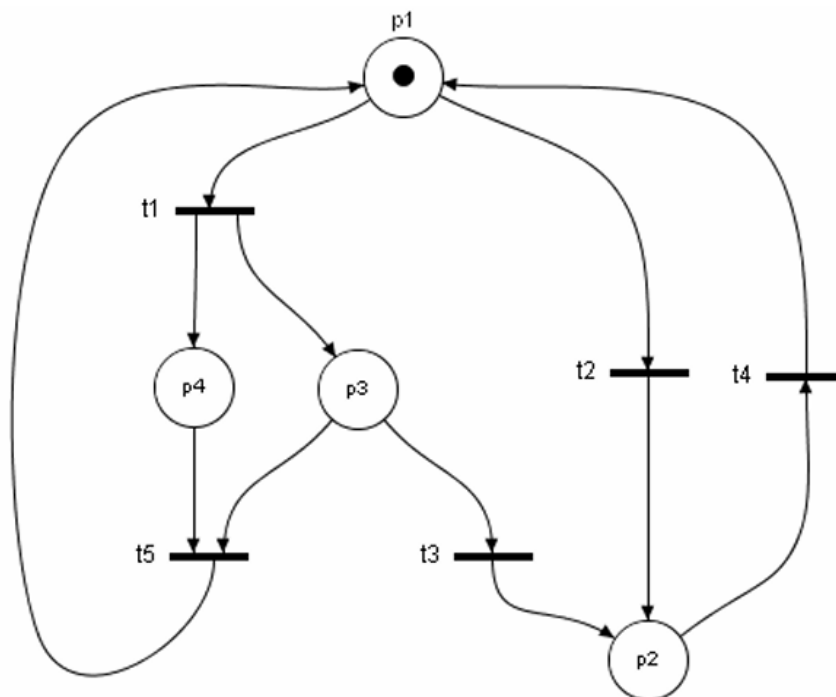


Fig. 1.1: Esempio Rete di Petri

L'evoluzione della rete è basata sull'uso di transizioni (t_1, t_2, t_3, t_4, t_5), operazioni che il sistema può effettuare, per spostare i tokens (risorse) fra le varie locazioni (p_1, p_2, p_3, p_4), stati interni della rete. Una transizione è abilitata se nel suo **preset** sono presenti sufficienti tokens richiesti dal suo utilizzo:

❖ **Preset (Definizione)**

Il preset di un posto/transizione y è dato da:

$$Pre(y) = \{z \in X \mid \langle z, y \rangle \in F\} \quad , \quad X = P \cup T$$

Osservando la figura, è possibile notare che a partire dalla marcatura iniziale $\{1,0,0,0\}$ sono abilitate a scattare le transizioni t_1 e t_2 . In assenza di un assegnamento di priorità, la risoluzione del conflitto è non deterministica. L'esecuzione di una transizione (comunemente detta "sparo") è un'operazione atomica, ciò vuol dire che i token vengono prelevati dal preset e depositati nel postset prima dello sparo di una qualsiasi altra transizione:

❖ **Postset (Definizione)**

Il postset di un posto/transizione y è dato da:

$$Post(y) = \{z \in X \mid \langle y, z \rangle \in F\} \quad , \quad X = P \cup T$$

Lo sparo di una transizione non consuma tempo in quanto esse appartengono alla categoria delle transizioni immediate. Tuttavia, è possibile introdurre la nozione di ritardo di sparo all'interno di una Rete di Petri tramite le cosiddette **Timed Petri Nets (TPN)**. Il concetto di tempo può essere associato ai posti (in questo caso le risorse rimangono immagazzinate per τ istanti di tempo) oppure alle transizioni. Quest'ultima è la soluzione più comune e concettualmente esprime il fatto che una transizione, da quando abilitata, spara dopo τ istanti di tempo a meno che essa non venga interrotta da una transizione a più alta priorità. All'interno della stessa rete possono sussistere sia transizioni immediate che temporizzate, in tal caso si parla di **GSPN (Generalized Stochastic Petri Nets)** in cui l'attributo stocastico si riferisce alla natura del ritardo τ , estratto mediante una distribuzione di probabilità esponenziale (utile per modellare la totale casualità del ritardo). A parità di abilitazione, sparano prima le transizioni immediate. Successivamente, se più transizioni temporizzate sono abilitate è possibile adottare due differenti strategie per risolvere i conflitti:

- a) Politica Race:** la transizione che spara per prima è quella avente ritardo τ minimo. A parità di ritardo, la scelta è non deterministica. Tale approccio risulta essere il più naturale in quanto pone in competizione le transizioni basandosi sul rispettivo ritardo. È importante notare che lo scatto di una transizione potrebbe causare la disabilitazione di qualche transizione con cui essa era precedentemente in conflitto:



Fig. 1.2: Disabilitazione della transizione t a carico di t'

Dall'immagine si evince che all'istante τ' la marcatura della rete è cambiata disabilitando la transizione t . In questi casi è possibile adottare due differenti approcci:

a.1) Politica Race con Memoria: si tiene traccia del tempo di sparo rimanente $\Delta = \tau - \tau'$. Alla successiva abilitazione la transizione t avrà ritardo di sparo pari a Δ .

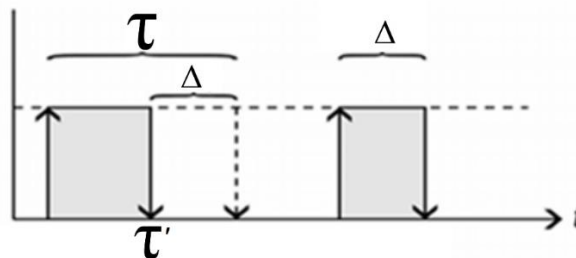


Fig. 1.3: Esempio politica race con memoria

a.2) Politica Race senza Memoria: non c'è traccia del tempo rimanente. Alla successiva abilitazione la transizione t necessiterà ancora di τ istanti di tempo prima di poter sparare.

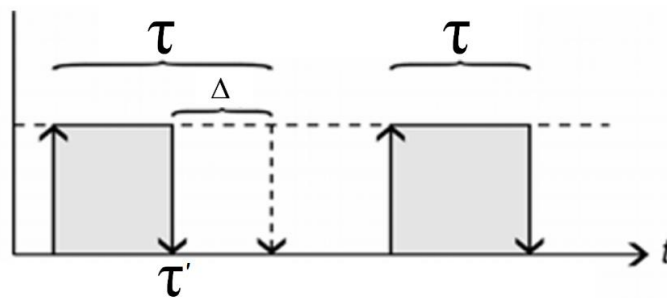


Fig. 1.4: Esempio politica race senza memoria

b) Politica Preselection: nell'istante in cui più transizioni temporizzate sono abilitate viene deciso a priori quale di queste è autorizzata a sparare. Così facendo, il conteggio del tempo di sparo avverrà solo per la transizione selezionata. La differenza rispetto alla politica race consiste nel fatto che le transizioni non entrano in competizione (attivando il proprio tempo di sparo), bensì solo la transizione selezionata inizia il conteggio.

1.1 Caso di Studio: CNC Machine

Il caso posto in analisi riguarda la progettazione di un semplice task svolto da una CNC Machine ((Computer Numerical Control). Una macchina CNC è una macchina operatrice in grado di completare diverse operazioni tipiche dell'industria come: taglio, assemblaggio e realizzazione di attività che richiedono un alto livello di precisione. La struttura tipicamente è composta dalla parte meccanica che rappresenta la geometria della macchina in termini di forma e dimensione. Tale componente consente il movimento del corpo grazie alla presenza di giunti traslazionali o rotazionali (ogni giunto identifica un grado di libertà). Vi è poi la parte terminale della macchina, comunemente detta End Effector. Solitamente, aggiunge ulteriori gradi di libertà alla macchina. La struttura dell'End Effector viene adattata alla specifica attività richiesta (taglio, pick and place di oggetti e così via). La figura seguente mostra un esempio di una moderna macchina CNC:



Fig. 1.1.1: Modello CNC machine Michelangelo

È possibile notare le due unità principali della macchina:

1. La stazione di lavoro: dedicata all'esecuzione del compito operativo e descritta finora.
2. L'unità di controllo: responsabile della gestione della generazione dei segnali di controllo per gli attuatori.

Da un punto di vista astratto, l'unità di controllo potrebbe essere identificata come il 'cervello' della macchina che agisce tramite il 'braccio' (la stazione di lavoro).

Il caso in esame prevede di svolgere un compito di taglio di una figura geometrica, in particolare un triangolo. La figura seguente illustra la traiettoria da compiere:

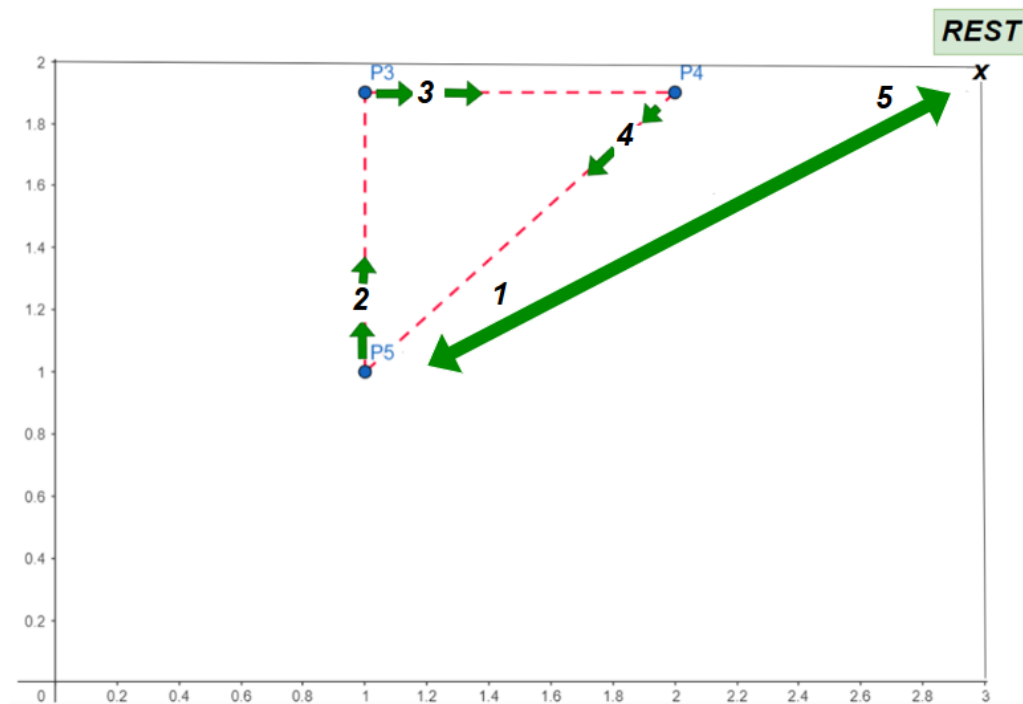


Fig. 1.1.2: Traiettoria da compiere

Sull'End Effector è posto un laser. La sequenza di azioni può essere riassunta come segue:

- 1.** A partire dalla posizione **Rest** il braccio meccanico deve raggiungere il punto **P5**.
- 2.** Accendere il Laser per prepararsi all'operazione di taglio.
- 3.** Effettuare la sequenza **P5→P3→P4→P5** per tagliare la figura geometrica.
- 4.** Spegner il Laser per prepararsi a tornare nella posizione **Rest**.
- 5.** Raggiungere **Rest**.

Effettuata tale sequenza, l'operazione può ritenersi completata.

La rappresentazione della corrispondente Rete di Petri è la seguente:

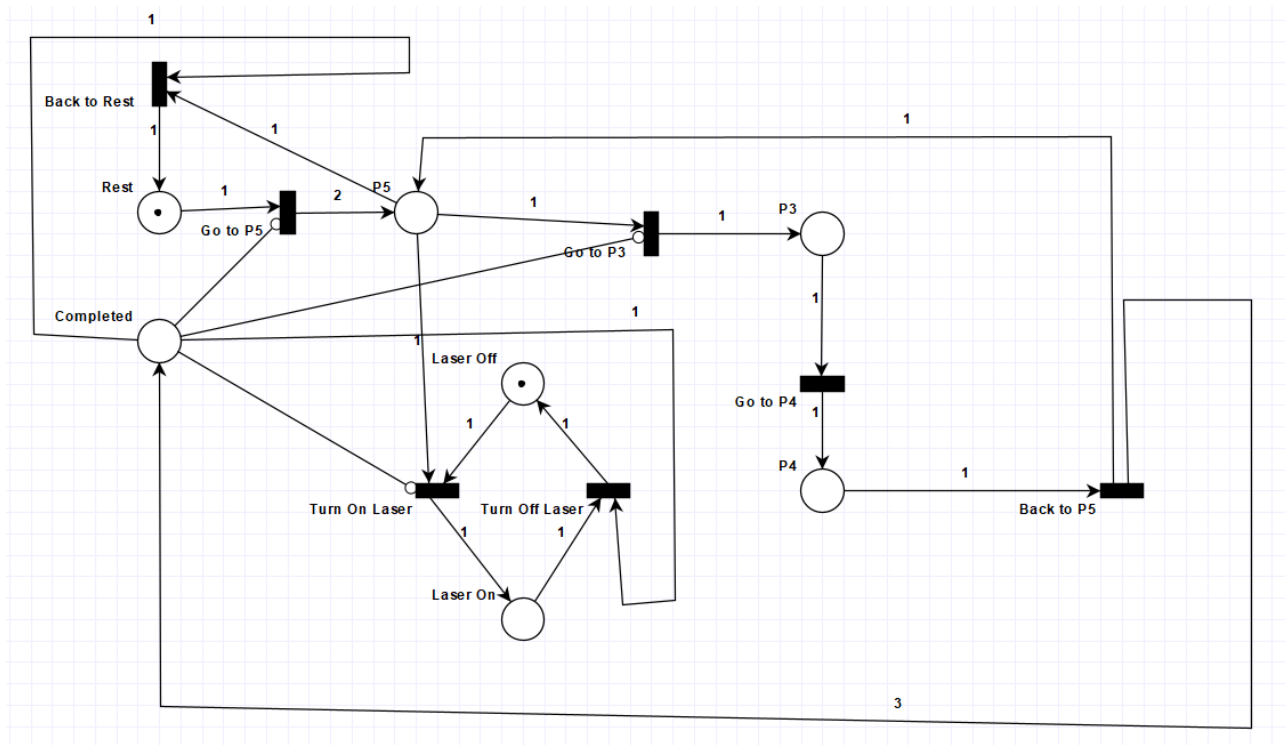


Fig. 1.1.3: Modello Rete di Petri (PIPE)

Il modello è stato ottenuto mediante il software Pipe, un utile strumento che permette di progettare Reti di Petri ed analizzarne importanti proprietà

Le transizioni **Turn On Laser** e **Turn Off Laser** hanno priorità maggiore rispetto alle altre. Per verificare che effettivamente il modello rispecchi il flusso di esecuzione voluto, analizziamo il grafo di raggiungibilità della rete. Esso è un grafo in cui i nodi rappresentano le marcature della rete e gli archi le transizioni che consentono di passare da una marcatura ad un'altra:

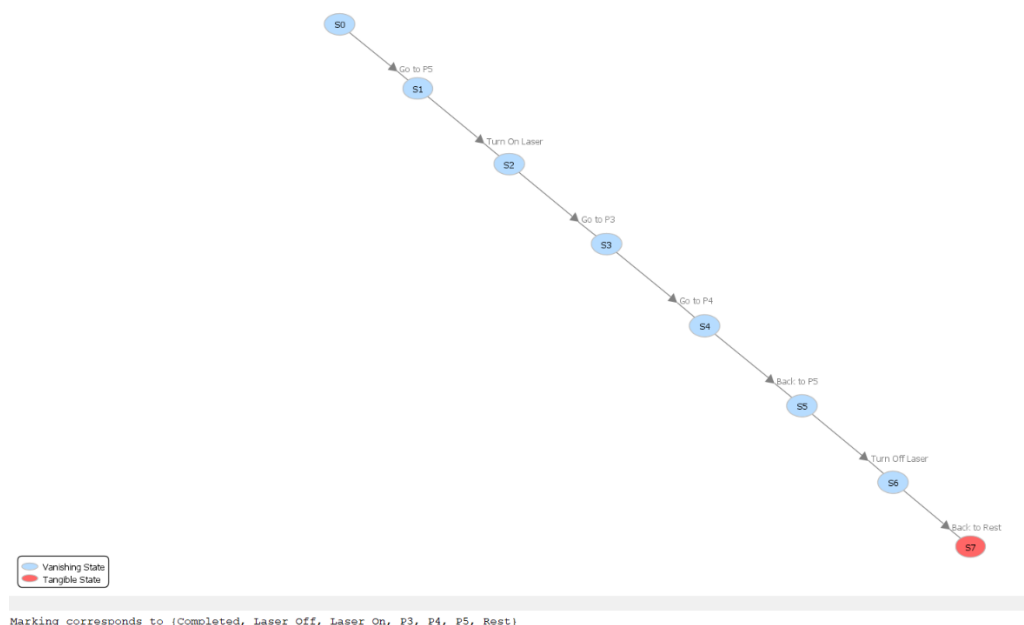


Fig. 1.1.4: Grafo ricoprente (PIPE)

Notiamo come effettivamente la rete evolva dallo stato **S0**:

$$\{0, 1, 0, 0, 0, 0, 1\}$$

Vale a dire il braccio si trova in posizione di **Rest** con il Laser spento, fino allo stato **S7**:

$$\{1, 1, 0, 0, 0, 0, 1\}$$

Ovvero, con tokens nei posti **Completed**, **Laser Off** e **Rest**. Negli istanti intermedi, la rete ha correttamente eseguito la transizione **Turn On Laser** solamente quando il braccio è arrivato in **P5** per poi definire la sequenza **P5→P3→P4→P5** con il laser acceso come richiesto.

A questo punto è possibile analizzare alcune proprietà tipiche delle Reti di Petri:

- **Deadlock**: situazione in cui l'evoluzione della rete si interrompe poiché nessuna transizione risulta abilitata per lo sparo. La rete in analisi è soggetta chiaramente a deadlock in quanto, una volta raggiunto lo stato finale, la presenza di un token in **Completed** inibisce tramite archi inibitori le uniche due possibili transizioni **Go to P5** e **Turn On Laser**. Tuttavia, tale situazione risulta essere programmata in quanto, completato il task, l'evoluzione della rete si può considerare conclusa.
- **Binarietà (Safeness)**: una rete è binaria se in ogni suo posto, per ogni possibile marcatura, il numero dei token è 0 oppure 1. La rete in analisi risulta essere non binaria poiché transizioni come **Go to P5** o **Back to P5** depositano un numero di token >1 nel loro postset.
- **Limitatezza (Boundedness)**: una rete è limitata di ordine k se in ogni marcatura raggiungibile il numero dei token in un qualsiasi posto non supera k. Analizzando la rete, è possibile osservare che **Back to P5** è la transizione che deposita il maggior numero di tokens. Pertanto, possiamo affermare che la rete è limitata di ordine 3.

❖ **Rappresentazione Matriciale**

Una Rete di Petri può essere descritta in forma matriciale, in modo da risultare più compatta e utile per analizzarne la dinamica. La rappresentazione è basata sull'utilizzo di tre matrici, di seguito estratte direttamente dalla rete:

1. Matrice di Input I: matrice di dimensione $|P| \times |T|$ con le seguenti entries:

$$I_{i,j} = \begin{cases} W(p_i, t_j) \\ 0 \text{ se arco non presente} \end{cases} \quad (1.1.1)$$

Con $W(p_i, t_j)$ peso dell'arco dal posto p_i alla transizione t_j .

	Back to P5	Back to Rest	Go to P3	Go to P4	Go to P5	Turn Off Laser	Turn On Laser
Completed	0	1	0	0	0	1	0
Laser Off	0	0	0	0	0	0	1
Laser On	0	0	0	0	0	1	0
P3	0	0	0	1	0	0	0
P4	1	0	0	0	0	0	0
P5	0	1	1	0	0	0	1
Rest	0	0	0	0	1	0	0

Fig. 1.1.5: Matrice di Input (PIPE)

2. **Matrice di Output O**: matrice di dimensione $|P| \times |T|$ con le seguenti entries:

$$O_{i,j} = \begin{cases} W(t_j, p_i) \\ 0 \text{ se arco non presente} \end{cases} \quad (1.1.2)$$

Con $W(t_j, p_i)$ peso dell'arco dalla transizione t_j al posto p_i .

	Back to P5	Back to Rest	Go to P3	Go to P4	Go to P5	Turn Off Laser	Turn On Laser
Completed	3	0	0	0	0	0	0
Laser Off	0	0	0	0	0	1	0
Laser On	0	0	0	0	0	0	1
P3	0	0	1	0	0	0	0
P4	0	0	0	1	0	0	0
P5	1	0	0	0	2	0	0
Rest	0	1	0	0	0	0	0

Fig. 1.1.6: Matrice di Output (PIPE)

3. **Matrice C**: matrice di dimensione $|P| \times |T|$:

$$C = O - I \quad (1.1.3)$$

	Back to P5	Back to Rest	Go to P3	Go to P4	Go to P5	Turn Off Laser	Turn On Laser
Completed	3	-1	0	0	0	-1	0
Laser Off	0	0	0	0	0	1	-1
Laser On	0	0	0	0	0	-1	1
P3	0	0	1	-1	0	0	0
P4	-1	0	0	1	0	0	0
P5	1	-1	-1	0	2	0	-1
Rest	0	1	0	0	-1	0	0

Fig. 1.1.6: Matrice C (PIPE)

2. Timed-Automata

Uppaal, applicativo sviluppato congiuntamente dall'Università di Uppsala ((Svezia) e Aalborg (Danimarca), rappresenta un utile strumento per l'implementazione e l'analisi di Timed Automata (TA). Un Timed Automata rappresenta un'estensione dei più comuni automi a stati finiti sviluppati da Mealy e Moore e formalmente è descritto dalla seguente sestupla:

$$TA = (L, L_0, C, A, E, I)$$

Dove:

- L : è un insieme di locazioni.
- L_0 : è la locazione iniziale.
- C : è l'insieme dei clock utilizzati.
- A : è un insieme di azioni.
- $E \subseteq L \times B(C) \times A \times 2^C \times L$: è un insieme di archi fra coppie di locazioni. Ogni arco ha una guardia, un'azione ed un sottoinsieme di clock.
- $I: L \rightarrow B(C)$: assegna invarianti alle locazioni.

La caratteristica principale rispetto agli automi a stati finiti standard consiste nell'introduzione del concetto di tempo denso attraverso l'uso dei clock. Pertanto, eventuali azioni dovranno tenere in considerazione non solo delle condizioni affinché possano realizzarsi, ma anche dei possibili vincoli temporali. Se ne deduce che l'utilizzo di un TA rappresenta una soluzione molto più conveniente per poter modellare fenomeni in cui la nozione di tempo svolge un ruolo essenziale.

2.1 Caso di Studio: Gestione Diga

La simulazione presa in esame è incentrata sulla gestione di una diga in cui si vuole mantenere il livello di acqua all'interno di un intervallo di sicurezza. Per far ciò, è presente un operatore che ha il compito di sorvegliare il livello corrente ed, eventualmente, intervenire per risolvere situazioni critiche che possono verificarsi. In particolare:

- Il livello di acqua **level** è ritenuto a norma se ricade nell'intervallo:

$$15[m] \leq level \leq 25[m]$$

- A causa di eventi atmosferici (pioggia) il livello di acqua può eccedere la soglia massima. In tal caso l'operatore deve intervenire tramite un segnale di comando che fa in modo di aprire una valvola di deflusso per permettere il ripristino delle condizioni di sicurezza. La valvola rimane aperta finché:

$$level > 25[m]$$

Una volta raggiunta la soglia di sicurezza (25[m]) l'operatore genera un segnale di comando per chiudere la valvola (non più necessaria).

- A causa di estesi periodi di siccità, il livello di acqua può scendere sotto la soglia minima. In tal caso l'operatore deve intervenire mediante un segnale di comando che fa aprire una pompa di prelievo da una riserva locale in modo da aumentare il livello corrente. La pompa rimane attiva finché:

$$level < 15[m]$$

Raggiunta la soglia minima (15[m]), l'operatore tramite un comando disattiva la pompa.

Il problema consiste, dunque, nella sincronizzazione fra i comandi impartiti dall'operatore e le operazioni di ripristino sulla diga.

A questo punto, procediamo con la realizzazione del sistema in Uppaal.

In prima istanza, è possibile riconoscere tre sottosistemi che operano nella simulazione:

1. **Operatore:** è il modulo che si occupa di implementare le azioni dell'operatore (invio di segnali di comando di apertura/chiusura e attivazione/disattivazione).
2. **Tank:** è il modulo in cui viene rappresentato il sistema della diga composto dalla valvola di deflusso e dalla pompa di prelievo.
3. **Environment:** è il modulo in cui viene simulata l'evoluzione del livello di acqua a causa degli eventi atmosferici.

Il sistema finale, dunque, sarà una composizione di tutti e tre i sottosistemi.

Nella figura seguente vengono dichiarate tutte le variabili di interesse per l'implementazione:

```
// Place global declarations here.
clock timer=0;
int level=23;
bool influx_completed=true, active=false, outflow_completed=true, open=false, rainy=true;
chan activate_pump, open_valve, close_valve, deactivate_pump;
```

Fig. 2.1.1: Istanziamento variabili (UPPAAL)

In cui:

- Il clock viene utilizzato come un timer per gestire i tempi di apertura/chiusura della valvola e attivazione/disattivazione della pompa.
- Il livello di acqua corrente è memorizzato in una variabile intera ed è inizializzato al valore 23 [m] (all'interno della soglia di sicurezza).
- Le variabili booleane servono a determinare se le operazioni interne alla diga sono state completate. Inoltre, grazie alla variabile **rainy** è possibile decidere se l'evento atmosferico è pioggia o siccità.
- I canali servono a sincronizzare le operazioni fra l'operatore e la diga (rappresentano dei segnali che possono assumere la semantica di input o output nel sistema).

Procediamo con l'implementazione dei sottosistemi:

➤ Environment

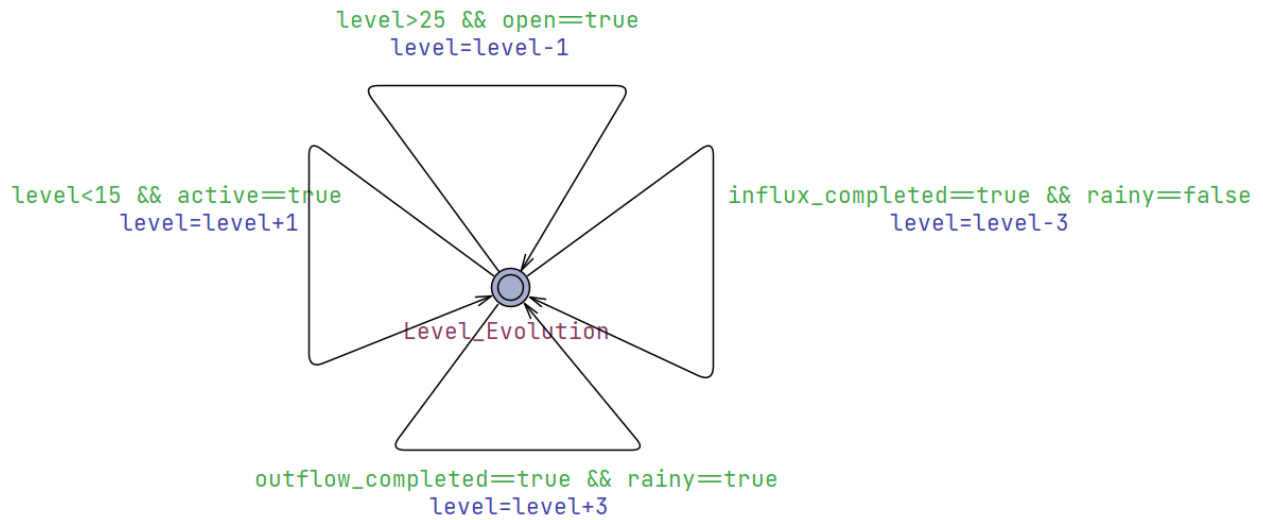


Fig. 2.1.2: Template Environment (UPPAAL)

Il sistema è composto da una sola locazione che rappresenta l'evoluzione del livello di acqua. In particolare, la rete evolve con la seguente logica:

- In condizioni di valvola e pompa non in funzione, il livello di acqua aumenta di 3 [m] al minuto se la giornata è piovosa, diminuisce di 3 [m] al minuto se il clima è secco.
- Quando la pompa è attiva il livello di acqua sale di 1 [m] al minuto.
- Quando la valvola è aperta il livello di acqua scende di 1 [m] al minuto.

I dati e le unità di misura scelti vanno considerati a titolo di esempio.

➤ Operator

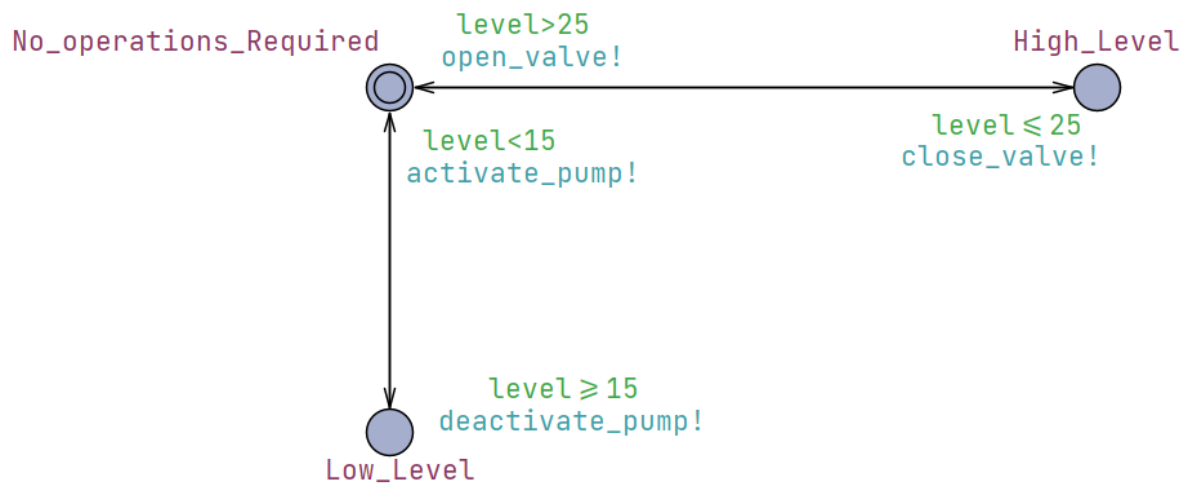


Fig. 2.1.3: Template Operator (UPPAAL)

L'operatore è descritto mediante tre locazioni. In particolare:

- Se il livello di acqua eccede il limite giunge sulla locazione critica **High_Level** impartendo il comando di apertura valvola. Una volta sceso il livello, rientra nella locazione di sicurezza chiudendo la valvola.
- Se il livello di acqua scende sotto la soglia minima giunge sulla locazione **Low_Level** attivando la pompa di prelievo. Ripristinato il livello, rientra nella locazione di sicurezza disattivando la pompa.

➤ Tank

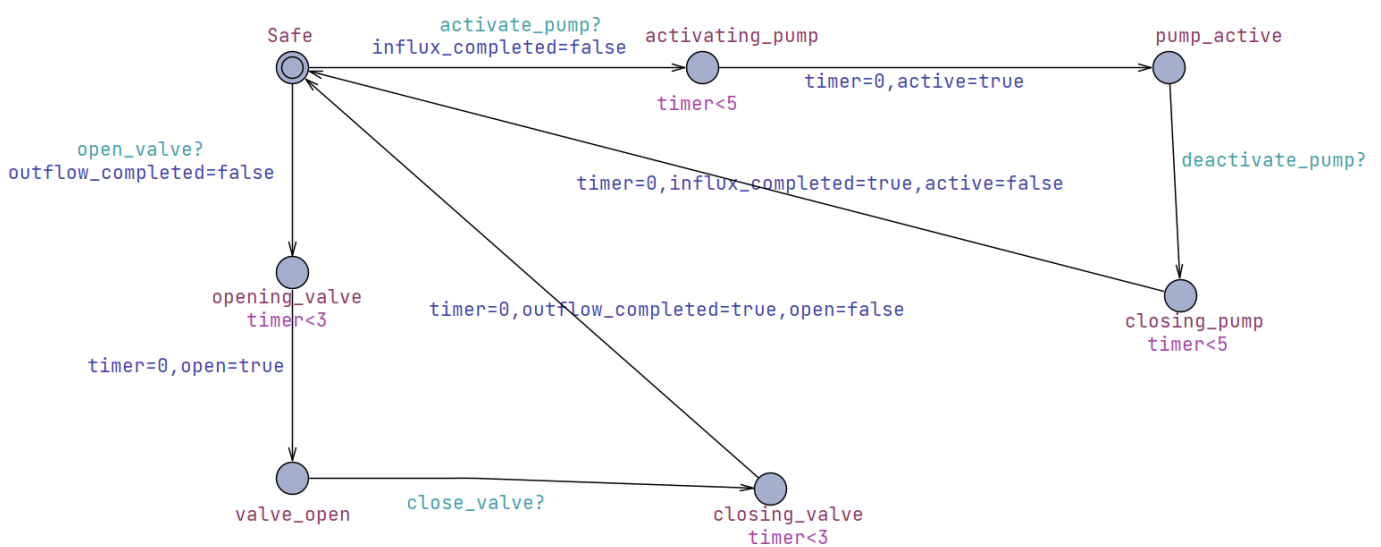


Fig. 2.1.3: Template Tank (UPPAAL)

La diga è composta dalla locazione di sicurezza (**Safe**) e dagli stati associati alle operazioni di apertura/chiusura valvola e attivazione/disattivazione pompa. In particolare:

- Giunta negli stati **activating_pump** e **closing_pump** la rete rimane in queste locazioni per 5 minuti tramite gli invarianti associati. Tale soluzione ha il compito di simulare lo scorrimento di tempo che intercorre fra il comando impartito dall'operatore e l'effettiva messa in opera del sistema meccanico.
- Ragionamento simile per quanto concerne le locazioni **opening_valve** e **closing_valve** in cui il tempo è impostato su 3 minuti.
- Gli invarianti sono associati alla variabile di clock **timer** il cui valore viene ripristinato a zero una volta terminate le operazioni di attivazione/disattivazione e apertura/chiusura.
- Ad ogni inizio o fine operazione di emergenza le rispettive variabili **influx_completed**, **outflow_completed**, **open** e **active** vengono aggiornate.

Definiti i singoli sottosistemi, il sistema finale, come anticipato, risulta essere la composizione dei tre, pertanto:

```

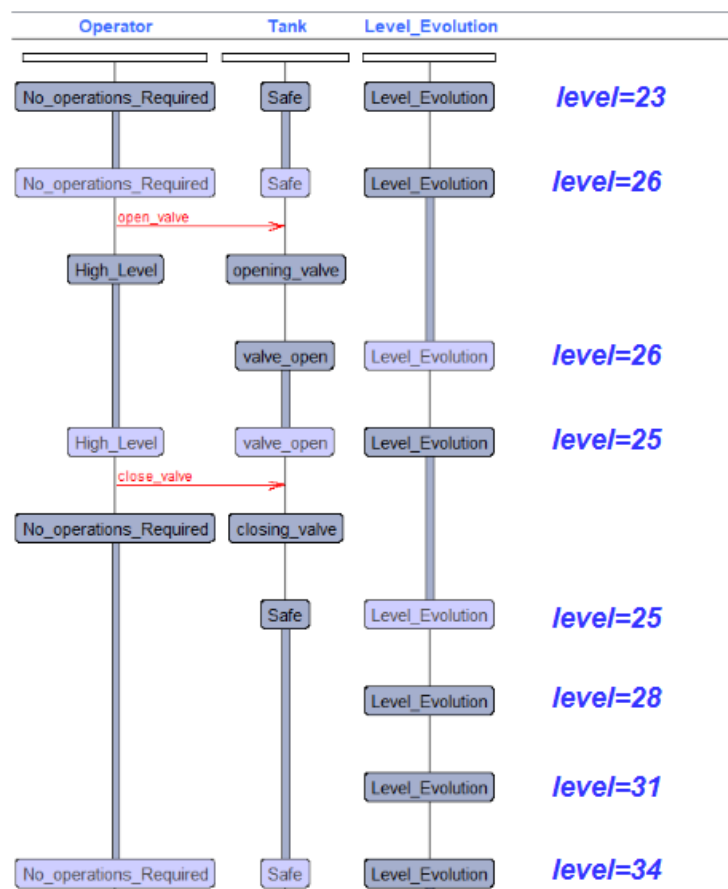
// Place template instantiations here.
Tank = Template_Tank();
Operator= Template_Operator();
Level_Evolution= Template_Environment();
// List one or more processes to be composed into a system.
system Operator,Tank,Level_Evolution;

```

Fig. 2.1.4: Composizione sistema complessivo (UPPAAL)

A questo punto è possibile procedere con una simulazione randomica tramite il software. Lo scenario prevede di implementare entrambe le condizioni ambientali: piovoso e secco.

a. Pioggia (rainy=true)



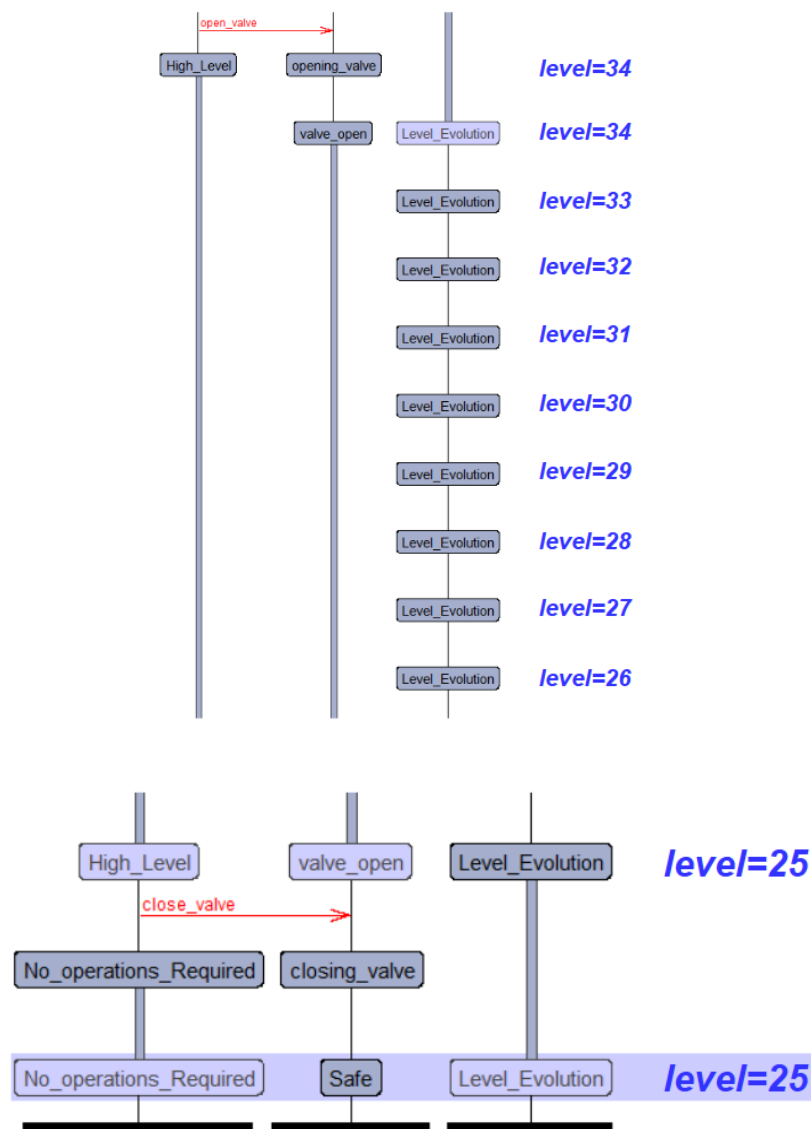


Fig. 2.1.5: Simulazione evento clima piovoso (UPPAAL)

I dati sul livello d'acqua sulla destra sono stati aggiunti per maggior chiarezza.

La simulazione mette in luce un aspetto fondamentale dell'implementazione. Si può notare, infatti, come il primo superamento del limite venga immediatamente notato dall'operatore che, prontamente, interviene aprendo la valvola permettendo, dunque, il ripristino immediato del livello. Tuttavia, il secondo superamento viene rilevato quando il livello è più alto rispetto al primo caso pur riuscendo, comunque, a riportare il sistema nella condizione di sicurezza. Ciò è dovuto al fatto che, quando il livello eccede il limite, i seguenti due scenari possono verificarsi:

1. L'operatore si accorge della situazione critica ed interviene.
2. Il livello continua a crescere con la pioggia.

Tale comportamento è una scelta progettuale in quanto si è voluto simulare il caso di un operatore 'distratto', mettendo in luce l'importanza di informatizzare i sistemi di controllo fin dalla fase di rilevazione (un sensore al posto dell'operatore avrebbe ottemperato al compito).

b. Siccità (rainy=false)

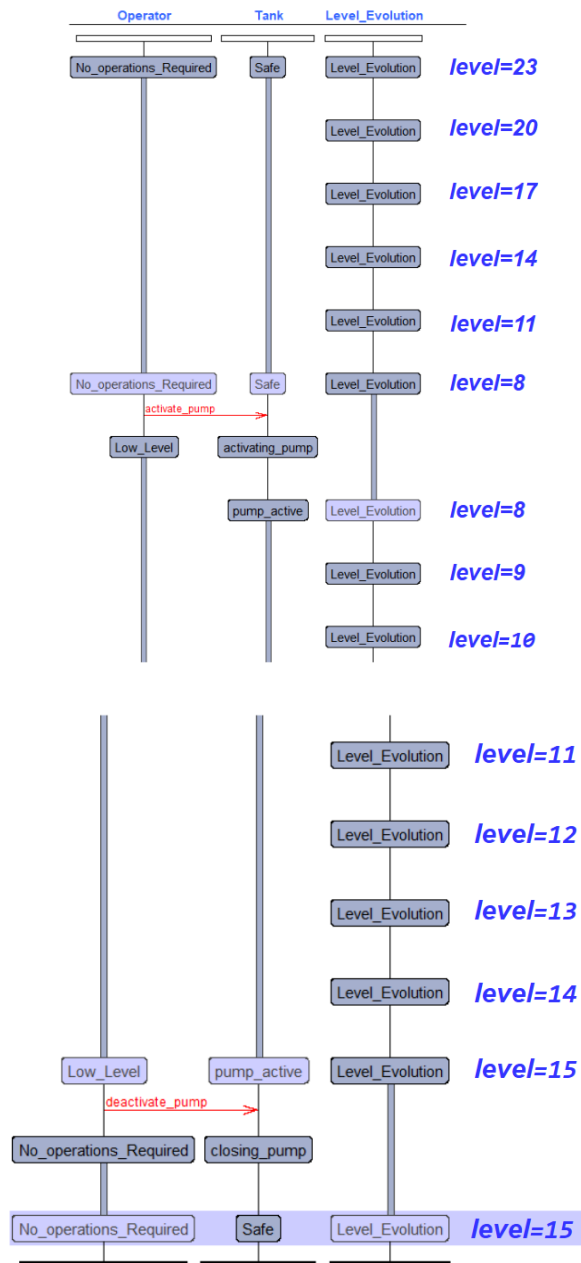


Fig. 2.1.6: Simulazione evento clima secco (UPPAAL)

Anche in questo caso è possibile notare che l'operatore è intervenuto quando metà della riserva era già esaurita.

Procediamo, in conclusione, con la sottomissione di alcune semplici interrogazioni riguardo alcune proprietà della rete:

```
A[] not deadlock
E[] Tank.valve_open imply Operator.High_Level
E[] rainy==true && Tank.pump_active
A<> Operator.Low_Level imply Tank.pump_active
```



Fig. 2.1.7: Query (UPPAAL)

È possibile notare che:

1. La rete è esposta a situazioni di deadlock. Di fatti, una volta che il livello rientra nell'intervallo di sicurezza se l'operatore, per qualsiasi motivo, non dovesse impartire i comandi di chiusura valvola o disattivazione pompa allora la rete entrerebbe in una situazione di blocco.
2. Esiste un percorso in cui se la valvola è aperta allora l'operatore si trova sulla locazione di rischio **High_Level**. Ciò appare in linea con la logica desiderata.
3. Non si verifica mai che se il clima è piovoso allora la pompa di prelievo è attiva. Tale interrogazione assicura che, in caso di livello in crescita, nella diga non venga mai introdotta ulteriore acqua.
4. Può verificarsi che se l'operatore si trova nella locazione **Low_Level** allora la pompa di prelievo è attiva. Notiamo come tale condizione non valga sempre. Come detto in precedenza, il comportamento simulato è quello di un operatore 'distratto'. In teoria, potrebbe non intervenire mai sulla diga, anche se molto improbabile.

3. Real-Time Scheduling

Lo scheduling rappresenta uno degli aspetti più importanti nell'analisi dei sistemi digitali. Nei paragrafi precedenti si è visto come poter modellare l'evoluzione di fenomeni più o meno complessi potendo dedurre importanti proprietà circa il loro funzionamento già dal modello progettato. Con la definizione di uno scheduling efficace si vuole far fronte alla problematica di dover organizzare l'elaborazione di più processi che vengono eseguiti su una risorsa hardware condivisa, ovvero la CPU. La gestione dello scheduling risulta di fondamentale importanza in tutti quei contesti in cui i tempi di esecuzione dei processi sono essenziali al fine di garantire un corretto funzionamento del sistema (sia esso soggetto a vincoli soft o hard).

Un processo viene definito come una sequenza di istruzioni. Al fine di poter organizzare più processi, l'asse temporale viene suddiviso in diversi frames (corrispondenti al **tempo di clock** del processore) ciascuno dei quali viene assegnato ad un processo in particolare per eseguire parte delle sue operazioni:

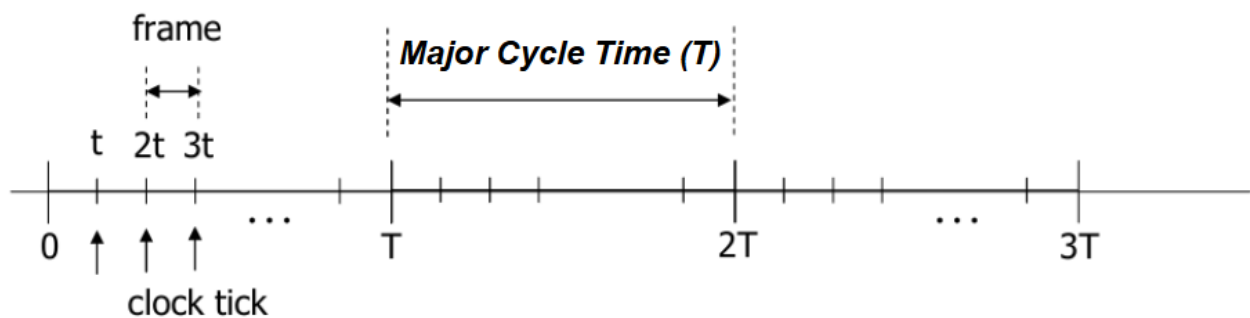


Fig. 3.1: Rappresentazione asse temporale e Major Cycle Time

Il **Major Cycle Time (MCT)** corrisponde al periodo che identifica dopo quanto tempo lo scheduling si ripete. Ciò vuol dire che, se non ci sono errori, al termine di ogni MCT ogni processo deve essere stato eseguito completamente almeno una volta.

Formalmente, un processo è identificato dalla seguente terna:

$$P = (C, T, D)$$

In cui:

- **C**: rappresenta il costo computazionale, ovvero il numero di frames di cui necessita il processo per essere eseguito completamente.
- **T**: rappresenta il periodo, ovvero dopo quanti frames il processo richiede nuovamente di essere processato (rilasciato). Noti i periodi di tutti i processi coinvolti, il MCT si può ottenere come segue:

$$MCT = m. c. m. (T_1, T_2, \dots, T_n) \quad n = \text{numero di processi}$$

- **D**: è la deadline, ovvero il tempo (unità di clock) relativo entro cui il processo deve essere eseguito da quando fa richiesta.

La classe di processi a cui siamo interessati è quella dei processi periodici (i tempi di rilascio sono descritti dal periodo, non c'è possibilità di rilasci consecutivi).

A questo punto, procediamo con l'analisi di alcuni dei più importanti algoritmi di scheduling.

Si considerino i seguenti tasks **indipendenti** da dover organizzare:

$$P_1 = (2, 5, 5) \quad P_2 = (1, 7, 7) \quad P_3 = (3, 10, 10)$$

L'attributo indipendente suggerisce che i processi non necessitano di dover accedere a risorse condivise fra loro.

Affinché sia possibile generare uno scheduling **fattibile**, ovvero in grado di rispettare i vincoli temporali di ogni processo, è importante analizzare il fattore di utilizzazione della CPU definito come:

$$U = \sum_{i=1}^n \frac{C_i}{T_i} \quad (3.1)$$

Con:

- n : pari al numero di processi.
- C_i : costo computazionale del processo i-esimo.
- T_i : periodo del processo i-esimo.

Applicato al caso di studio si ottiene:

$$U = \frac{2}{5} + \frac{1}{7} + \frac{3}{10} = 0.8429$$

In prima istanza possiamo affermare che poiché risulta:

$$U < 1$$

La condizione necessaria affinché uno schedule sia fattibile (indipendentemente dall'algoritmo scelto) risulta verificata.

3.1 Rate Monotonic (RM)

L'algoritmo Rate Monotonic (**RM**) assegna staticamente priorità (π) ai processi in modo inversamente proporzionale ai periodi: più breve è il periodo, maggiore è la priorità. RM si configura come un algoritmo pre-emptivo, ovvero l'esecuzione di un processo può essere interrotta al successivo clock qualora un processo a più alta priorità dovesse chiedere il rilascio. Nel caso dell'esempio in questione si ha:

$$\pi(P_1) > \pi(P_2) > \pi(P_3)$$

In quanto P_1 risulta avere il periodo più breve.

❖ **Bound LL**

Liu e Layland hanno provato che sotto RM uno schedule è certamente fattibile (condizione sufficiente) se:

$$U \leq n * \left(2^{\frac{1}{n}} - 1\right) \quad 3.1.1$$

Applicato al caso di studio si ha:

$$0.8429 \leq 0.779$$

Poiché la disuguaglianza non è rispettata, la condizione sufficiente non è verificata. Tuttavia, in precedenza è stato mostrato che i processi rispettano la condizione necessaria, pertanto, uno scheduling sotto RM potrebbe comunque esistere. Per accertare l'effettiva possibilità di organizzare i processi, facciamo riferimento all'**analisi esatta** di **Joseph e Pandya** :

❖ **Analisi Esatta (Joseph e Pandya)**

Detto R_i il Response Time (tempo di risposta), ossia il worst case time richiesto dal task i-esimo per completare la propria esecuzione, si ha:

$$R_i^{n+1} = C_i + \sum_{\forall j \in hp(i)} \left\lceil \frac{R_i^n}{T_j} \right\rceil * C_j \quad (3.1.2)$$

In cui:

- C_i : è il costo computazionale del processo i-esimo su cui si sta conducendo il calcolo del rispettivo R_i .
- $hp(i)$ rappresenta l'insieme dei tasks j a più alta priorità rispetto al task i-esimo.
- T_j : è il periodo del task j-esimo.
- C_j : è il costo computazionale del task j-esimo.
- $[x]$: identifica l'operatore soglia che restituisce il minimo intero maggiore o uguale ad x.
- $R_i^0=0$: inizializzazione.

L'analisi si basa sull'iterazione dell'equazione ricorsiva [3.1.2](#) fin quando si giunge a convergenza, ovvero:

$$R_i^{n+1} = R_i^n$$

Il valore trovato di R_i si confronta quindi con la deadline D_i . Per la schedulabilità deve valere:

$$R_i \leq D_i \quad \forall i = 1, \dots, \text{numero dei processi}$$

Per accertarci della possibilità di ottenere uno schedule sotto RM, procediamo con i calcoli dei tempi di risposta dei processi in questione ricordando che vale:

$$\pi(P_1) > \pi(P_2) > \pi(P_3)$$

1. Tempo di Riposta per P_1

$$R_1 = 2$$

2. Tempo di Riposta per P_2

$$R_2^1 = 1 + \left\lceil \frac{R_2^0}{T_1} \right\rceil * C_1 = 1$$

$$R_2^2 = 1 + \left\lceil \frac{R_2^1}{T_1} \right\rceil * C_1 = 3$$

$$R_2^3 = 1 + \left\lceil \frac{R_2^2}{T_1} \right\rceil * C_1 = 3$$

3. Tempo di Riposta per P_3

$$R_3^1 = 3 + \left\lceil \frac{R_3^0}{T_2} \right\rceil * C_2 + \left\lceil \frac{R_3^0}{T_1} \right\rceil * C_1 = 3$$

$$R_3^2 = 3 + \left\lceil \frac{R_3^1}{T_2} \right\rceil * C_2 + \left\lceil \frac{R_3^1}{T_1} \right\rceil * C_1 = 6$$

$$R_3^3 = 3 + \left\lceil \frac{R_3^2}{T_2} \right\rceil * C_2 + \left\lceil \frac{R_3^2}{T_1} \right\rceil * C_1 = 8$$

$$R_3^4 = 3 + \left\lceil \frac{R_3^3}{T_2} \right\rceil * C_2 + \left\lceil \frac{R_3^3}{T_1} \right\rceil * C_1 = 9$$

$$R_3^5 = 3 + \left\lceil \frac{R_3^4}{T_2} \right\rceil * C_2 + \left\lceil \frac{R_3^4}{T_1} \right\rceil * C_1 = 9$$

Giunti a convergenza tutti i tempi di risposta, procediamo con il controllo:

$$\begin{cases} R_1 \leq D_1 \\ R_2 \leq D_2 \\ R_3 \leq D_3 \end{cases} \rightarrow \begin{cases} 2 \leq 5 \\ 3 \leq 7 \\ 9 \leq 10 \end{cases}$$

Poiché tutte le disuguaglianze risultano verificate, possiamo affermare che i processi soddisfano i requisiti dell'analisi esatta di Joseph e Pandya, pertanto, sono schedabili sotto RM nonostante non abbiano superato il test sul bound LL.

Per organizzare i processi, facciamo utilizzo del software **Matlab** tramite il suo modulo **Task Scheduler**. Il risultato fornito è il seguente:

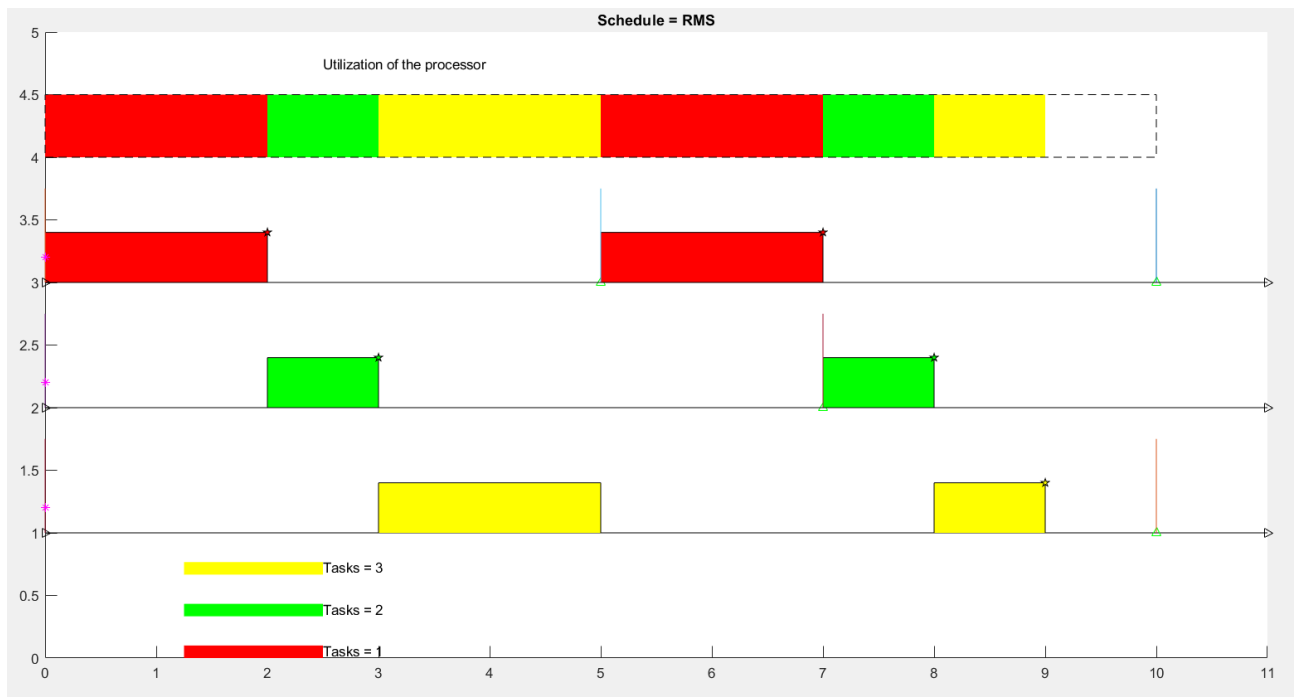


Fig. 3.1.1: Schedule Rate Monotonic (Matlab)

È possibile notare come effettivamente esista uno scheduling. Gli asterischi in alto sulla destra identificano l'istante in cui il processo termina l'esecuzione; pertanto, si può osservare come all'interno del MCT i processi P_1 e P_2 siano stati eseguiti due volte mentre P_3 una volta sola. Ciò è in accordo con i rispettivi periodi e le priorità assegnate dall'algoritmo.

3.2 Earliest Deadline First (EDF)

L'algoritmo EDF assegna le priorità dinamicamente. Tra i processi ready, quello con la deadline più imminente ha la massima priorità e pre-empta il processo in esecuzione. Nel caso di EDF vale la seguente proprietà:

$U \leq 1$ è condizione necessaria e sufficiente affinché uno schedule sia fattibile

Pertanto, l'algoritmo EDF si configura come **ottimale**, ovvero è in grado di generare uno schedule fattibile se questo esiste.

Nel caso di interesse si ottiene:

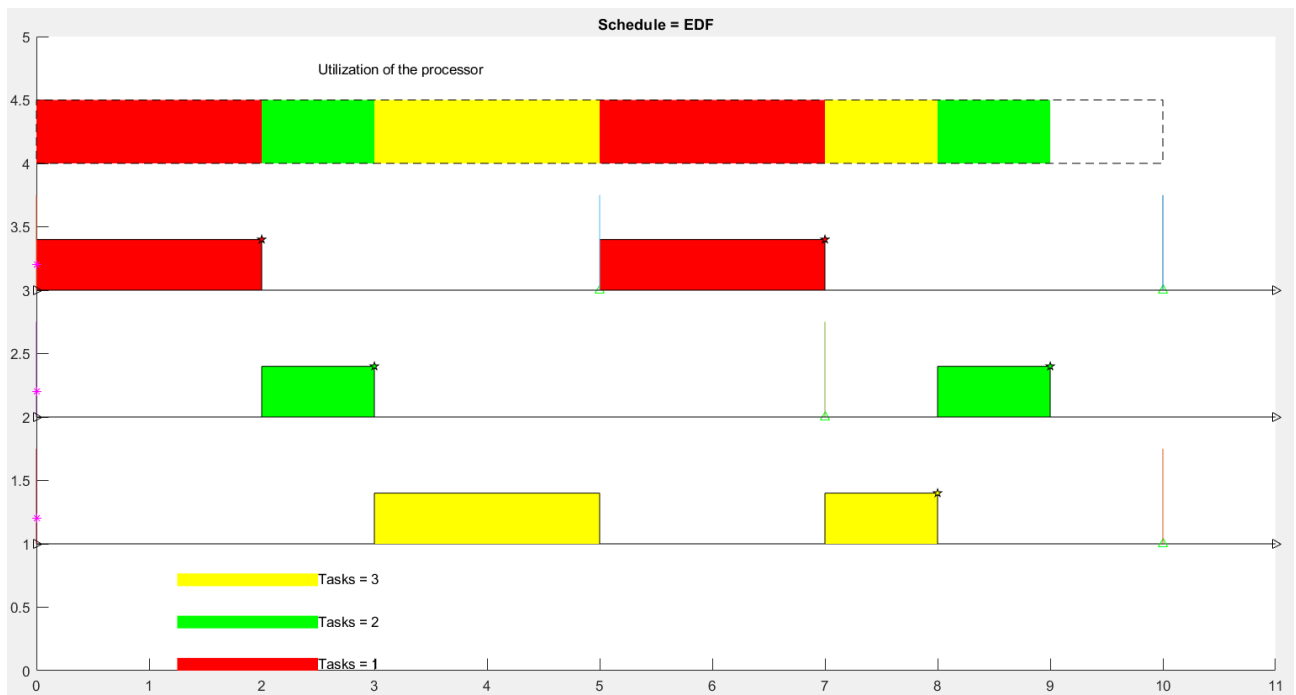


Fig. 3.2.1: Schedule Earliest Deadline First (Matlab)

Inizialmente i processi vengono organizzati come nel caso precedente poiché:

$$D_1 < D_2 < D_3 \rightarrow \pi(P_1) > \pi(P_2) > \pi(P_3)$$

Per notare la differenza rispetto all'algoritmo RM, poniamo l'attenzione all'istante 7:

- P_1 ha appena terminato l'esecuzione ed il prossimo rilascio avverrà all'istante:

$$7 + T_1 = 12$$

- P_2 effettua una seconda richiesta di rilascio in quanto il suo periodo è appena scaduto. La deadline assoluta corrispondente è pari a:

$$7 + D_2 = 14.$$

- P_3 non ha ancora terminato l'esecuzione e la sua deadline assoluta è pari a:

$$0 + D_3 = 10$$

Dunque, l'algoritmo impone $\pi(P_3) > \pi(P_2)$ permettendo di rispettare i vincoli temporali.

3.3 Least Slack Time (LST)

L'algoritmo LST si basa sul confronto di una variabile detta **tempo di slack**(*st*) definita come:

$$st = (D - t) - C' \quad (3.3.1)$$

Con:

- **D**: deadline del processo.
- **t**: tempo reale relativo all'inizio del ciclo.
- **C'**: porzione del corpo del processo non ancora eseguita (frames rimanenti).

Ad ogni istante si controlla la variabile *st* associata ad ogni task. Il processo con minimo tempo di slack ha la massima priorità e può pre-emptare sui restanti. Di seguito viene riportato lo schedule corrispondente:

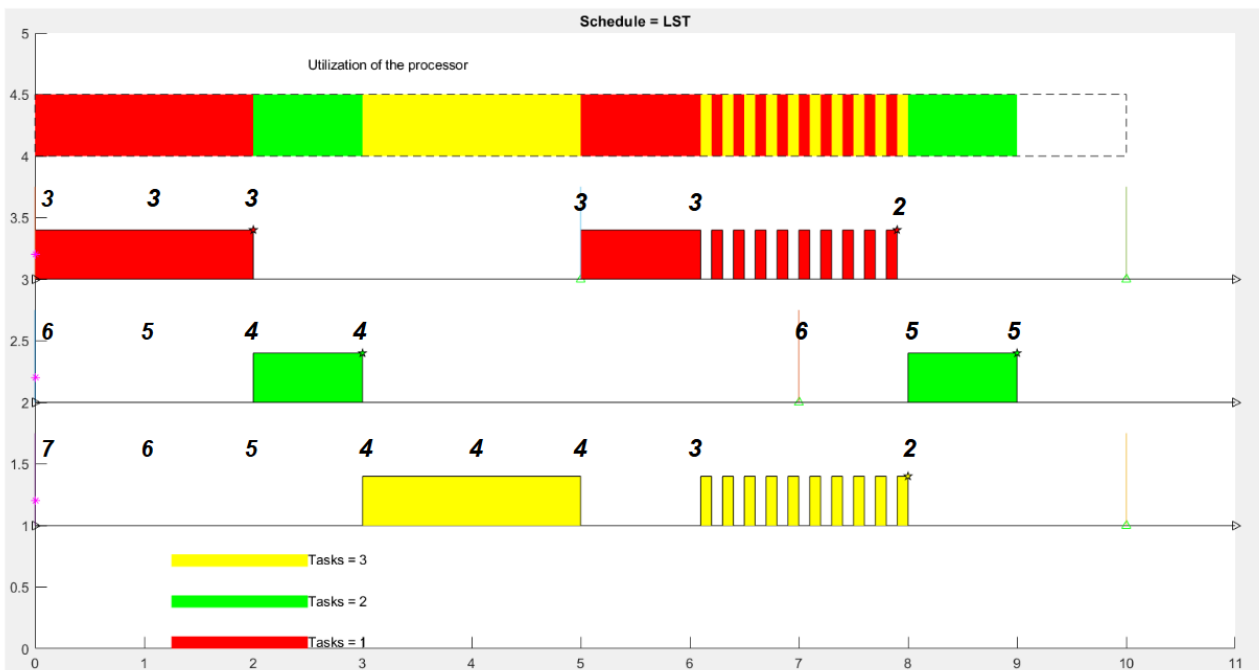


Fig. 3.3.1: Schedule Least Slack Time (Matlab)

Nel grafico sono stati aggiunti i valori di slack dei processi ad ogni istante di tempo. Inizialmente:

$$st(P_1) = (5 - 0) - 2 = 3, \quad st(P_2) = (7 - 0) - 1 = 6, \quad st(P_3) = (10 - 0) - 3 = 7$$

Dunque: $\pi(P_1) > \pi(P_2) > \pi(P_3)$.

All'istante 5 P_2 è stato eseguito e farà una nuova richiesta al tempo $T_2 = 7$. Per quanto riguarda P_1 e P_3 :

$$st(P_1) = (10 - 5) - 2 = 3, \quad st(P_3) = (10 - 5) - 1 = 4$$

Il processo P_1 effettua una nuova richiesta di rilascio e poiché il suo slack time è minore di quello del processo P_3 preempta.

Dall'istante 6 all'istante 8 P_1 e P_3 condividono lo stesso slack time. Il software decide di alternare la loro esecuzione in quanto appare evidente che il confronto adoperato per assegnare le priorità è basato sul $\leq$ anziché il semplice $<$. Notiamo come all'istante 7 la nuova richiesta di rilascio di P_2 non venga accolta, coerentemente con la politica dell'algoritmo.